

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования

«Российский государственный университет

им. А.Н. Косыгина (Технологии. Дизайн. Искусство)»

Кафедра Информационных технологий и компьютерного дизайна

Отчет по курсовой работе

по дисциплине «Интеллектуальные информационные системы и технологии»

по теме: **«Разработка веб-приложения для координации и синхронизации
данных съёмочной группы кинопроекта»**

Выполнил: Войтенко Степан Игоревич,
гр. МИМИ-122

Проверила: Каршакова Л.Б.

Москва 2026

ОГЛАВЛЕНИЕ

Введение	3
ГЛАВА 1. Теоретические основы автоматизации управления	
кинопроизводством	5
1.1 Информационное взаимодействие в съёмочной группе	5
1.2 Обзор существующих систем управления кинопроизводством	7
1.3 Веб-технологии как платформа для многопользовательских ИС	9
1.4 Безопасность данных: криптографические методы защиты паролей	11
1.5 Многоязычные информационные системы и методы интернационализации	13
ГЛАВА 2. Проектирование и разработка системы SyncHub	15
2.1 Анализ требований к системе	15
2.2 Архитектура приложения	17
2.3 Реализация алгоритма хеширования и верификации паролей	19
2.4 Реализация системы мультязычного интерфейса (i18n)	23
2.5 Вспомогательные алгоритмы системы	27
ГЛАВА 3. Тестирование и оценка эффективности	30
3.1 Сценарии тестирования	30
3.2 Оценка эффективности	32
Заключение	34
Список используемых источников	36
Приложение А. Листинги ключевых алгоритмов	38

ВВЕДЕНИЕ

Современное кинопроизводство предполагает одновременную работу нескольких специалистов: сценариста, режиссёра, монтажёра, костюмера, визажиста и звукорежиссёра. Каждый из них ведёт собственный раздел проекта, однако данные этих разделов неразрывно связаны между собой — изменение сценария влечёт пересмотр раскадровки, корректировку костюмов и тайм-кода монтажа.

На практике взаимодействие участников таких команд организуется через электронную почту, мессенджеры и разрозненные таблицы. Это приводит к задержкам, возникновению конфликтующих версий данных и потере актуальной информации при финальной сборке материала.

Актуальность настоящей работы обусловлена отсутствием специализированного отечественного веб-инструмента, объединяющего рабочие пространства всех ключевых участников съёмочной группы в единой системе с поддержкой синхронизации данных и работы в условиях нестабильного интернет-соединения.

Целью данной работы является разработка информационной системы SyncHub — многопользовательского веб-приложения для совместной работы съёмочной группы, обеспечивающего хранение, защиту и отображение данных каждой роли в рамках единого проекта.

Для достижения поставленной цели решались следующие задачи:

- проанализировать существующие системы управления кинопроизводством и выявить функциональные пробелы в доступных аналогах;
- изучить архитектурные подходы к построению многопользовательских веб-систем;
- разработать и реализовать алгоритм безопасного хеширования паролей с защитой от brute-force и timing-атак;
- спроектировать и реализовать систему интернационализации интерфейса (i18n) с поддержкой русского, английского и китайского языков;

— провести тестирование разработанной системы и оценить её практическую эффективность.

Объектом исследования является процесс информационного взаимодействия участников съёмочной группы в рамках кинопроекта.

Предметом исследования выступают методы и алгоритмы разработки защищённой многопользовательской веб-системы с поддержкой многоязычного интерфейса.

Практическая значимость работы заключается в том, что разработанная система используется в качестве дипломного проекта и может применяться реальными командами кинопроизводства без установки дополнительного программного обеспечения, поскольку работает полностью в браузере.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АВТОМАТИЗАЦИИ УПРАВЛЕНИЯ КИНОПРОИЗВОДСТВОМ

Глава раскрывает теоретический фундамент разрабатываемой системы. Последовательно рассматриваются специфика информационных потоков в съёмочной группе, обзор существующих программных решений для управления кинопроизводством, обоснование выбранного веб-стека технологий, а также теоретические основы двух ключевых алгоритмических компонентов системы: криптографического хеширования паролей и многоязычной поддержки интерфейса.

1.1 Информационное взаимодействие в съёмочной группе

Кинопроизводство представляет собой сложный коллективный процесс, в который вовлечено несколько профильных специалистов с различной зоной ответственности. Шесть ключевых ролей — сценарист, режиссёр, костюмер, визажист, монтажёр и звукорежиссёр — формируют уникальный состав данных. Сценарист работает с текстом сценария и репликами; режиссёр фиксирует постановочные пометки, мизансцены и раскадровку; костюмер ведёт реестр костюмов и привязок к персонажам; визажист — описание грима и схемы лица; монтажёр — тайм-код и порядок склеек; звукорежиссёр — звуковые маркеры и реплики синхронного звука. Несмотря на различие в характере данных, все эти разделы образуют единое информационное пространство проекта.

Между разделами существует устойчивая зависимость. Сценарий определяет состав сцен и реплик, на основе которых строится раскадровка; раскадровка задаёт визуальную структуру, на которую опирается монтаж; монтаж формирует тайм-код, к которому привязываются звуковые маркеры и пометки костюмера. Изменение в любом узле этой цепочки требует пересмотра всех зависимых разделов. Сильнейшая особенность таких данных — общая временная ось: подавляющее большинство пометок

участников группы привязаны к конкретной позиции на хронометраже сцены или эпизода.

На практике обмен этими данными чаще всего ведётся через мессенджеры, электронную почту и обособленные таблицы. Подобная организация порождает три типичные проблемы: потеря актуальности из-за параллельных правок, конфликты версий при отсутствии единого хранилища и задержки на этапе финальной сборки материала, когда требуется собрать пометки от всех участников. Каждая из перечисленных проблем напрямую снижает скорость и качество кинопроизводственного процесса.

Таким образом, перечисленные особенности — высокая степень специализации, жёсткие межролевые зависимости и общая временная ось данных — обуславливают необходимость единой информационной системы с ролевыми рабочими местами и централизованным хранилищем, что подтверждает целесообразность разработки системы SyncHub в данной работе.

1.2 Обзор существующих систем управления кинопроизводством

На рынке представлен ряд систем для управления кинопроизводством. Для анализа выбраны наиболее распространённые инструменты: Cerebro, Celtx и StudioBinder.

Cerebro — отечественная профессиональная система управления проектами в области кино и анимации, разработанная для крупных студий. Платформа охватывает планирование задач, контроль рабочего времени, версионирование медиафайлов и коммуникацию между отделами. Cerebro распространяется по платной подписке, ориентирован на среду студийных рабочих станций и не предполагает специализированных рабочих мест костюмера и визажиста. Высокая стоимость и сложность развёртывания делают систему недоступной для малых съёмочных групп и учебных проектов.

Celtx — облачный сервис, изначально позиционируемый как инструмент сценариста и расширенный до функций предварительного

планирования съёмок: раскадровка, шот-лист, разбивка сцен. Celtx предлагает частично бесплатный план с ограничением на число проектов и пользователей; полный функционал доступен только в платной подписке. Интерфейс системы реализован исключительно на английском языке, что затрудняет применение в русскоязычных коллективах. Роли костюмера и визажиста как самостоятельные рабочие пространства в Celtx отсутствуют.

StudioBinder — американский веб-сервис для управления продюсерским процессом, ориентированный на коммерческую видеопroduкцию: рекламные ролики, музыкальные клипы, короткометражные фильмы. Платформа включает шот-листы, шедулинг, генератор дневных планов съёмок и коммуникацию по проекту. StudioBinder распространяется по платной подписке, не имеет русскоязычного интерфейса и не поддерживает работу при отсутствии интернет-соединения. Анатомические карты для костюмера и визажиста в системе не реализованы.

Для наглядного сопоставления функциональных возможностей рассматриваемых систем составлена сравнительная таблица (таблица 1).

Таблица 1 — Сравнение систем управления кинопроизводством

Критерий	SyncHub	Cerebro	Celtx	StudioBinder
Русский интерфейс	Да	Да	Нет	Нет
Бесплатный план	Да (полный)	Нет	Частично	Нет
Роль «Костюмер»	Да	Нет	Нет	Нет
Роль «Визажист»	Да	Нет	Нет	Нет
Офлайн-режим	Да	Нет	Нет	Нет
Не требует установки	Да	Нет	Частично	Да
Анатомическая карта тела	Да	Нет	Нет	Нет
Тайм-код маркеры	Да	Да	Нет	Нет

Сравнительный анализ показывает, что ни один из рассмотренных аналогов не охватывает одновременно все потребности съёмочной группы: специализированные рабочие места костюмера и визажиста с анатомическими картами тела и лица отсутствуют во всех проанализированных системах. Большинство аналогов распространяются по платной подписке, что ограничивает их применимость в малых проектах и учебной деятельности. Кроме того, ни один из них не поддерживает офлайн-режим работы, критически важный при съёмках в локациях с нестабильным интернет-соединением.

Таким образом, существующие решения имеют существенные ограничения с точки зрения полноты ролевого охвата и доступности, что подтверждает целесообразность разработки системы SyncHub в рамках данной работы.

1.3 Веб-технологии как платформа для многопользовательских ИС

Выбор технологического стека для системы SyncHub определяется требованиями к интерактивности интерфейса, типобезопасности кода и минимизации внешних зависимостей. В качестве базовой клиентской библиотеки применяется React 18, обеспечивающий компонентную модель построения интерфейса и декларативный подход к управлению состоянием. Компонентная архитектура соответствует ролевой структуре приложения: каждая вкладка проекта (сценарий, режиссёрская, костюмы, визаж, монтаж, звук) реализуется в виде отдельного React-компонента с локальным состоянием.

Дополнением React выступает TypeScript — надмножество JavaScript со статической системой типов. Использование TypeScript критично для проекта SyncHub, поскольку он оперирует сложными доменными типами: Project, Marker, BodySilhouette, StoryboardGrid, Character. Статическая типизация обеспечивает обнаружение значительной части ошибок на этапе компиляции, а не во время выполнения, что особенно ценно при работе со сложно вложенными структурами данных кинопроекта.

В качестве инструмента сборки выбран Vite 5 — современный сборщик с поддержкой нативных ES-модулей в режиме разработки. Особенностью Vite является механизм горячей замены модулей (Hot Module Replacement, HMR), позволяющий сохранять состояние компонентов при изменении исходного кода, что существенно ускоряет цикл разработки сложного интерфейса с многочисленными вкладками и состояниями.

Серверная часть реализована на платформе Node.js с применением фреймворка Express.js. Однопоточная событийная модель Node.js эффективна для приложений, в которых преобладают операции ввода-вывода (чтение и запись JSON-файла, передача данных по сети) — характерная нагрузка для системы координации съёмочной группы. Express.js предоставляет минимальный, но достаточный набор средств маршрутизации и работы с промежуточным программным обеспечением (middleware), что соответствует учебному масштабу проекта.

В качестве хранилища данных используется JSON-файл, расположенный на стороне сервера. Это решение обосновано целевыми характеристиками прототипа: нулевые внешние зависимости, простота переноса между средами, читаемость данных без специальных инструментов и достаточная производительность при ожидаемой нагрузке порядка десятков параллельных пользователей. Для глобального состояния клиентской части (аутентификация, текущий язык интерфейса) применяется React Context API — встроенный механизм передачи данных без явного «пробрасывания» свойств (props drilling) через все промежуточные компоненты.

Таким образом, выбранный стек React + TypeScript + Vite + Node.js + Express обеспечивает баланс между функциональностью, безопасностью и минимизацией зависимостей, что соответствует требованиям, предъявляемым к учебному дипломному проекту.

1.4 Безопасность данных: криптографические методы защиты паролей

Многопользовательские системы, хранящие аккаунты пользователей, обязаны обеспечивать защиту паролей от компрометации даже в случае утечки базы данных. Наивное хранение паролей в открытом виде или в виде MD5/SHA-1 хеша недопустимо: злоумышленник, получивший доступ к базе, может восстановить пароли с помощью заранее предвычисленных таблиц (rainbow tables) или перебора.

Для противодействия таким атакам применяются специализированные функции формирования ключа (Key Derivation Function, KDF): bcrypt, Argon2, scrypt. В отличие от криптографических хеш-функций общего назначения, KDF разработаны с явной целью замедлить вычисление — каждый вызов функции намеренно требует значительного количества процессорного времени и памяти.

Алгоритм scrypt, предложенный Колином Персивалом в 2009 году, обладает параметрически настраиваемой «жесткостью» (memory-hardness): он требует не только процессорных, но и значительных объемов оперативной

памяти, что делает атаку с использованием специализированных устройств (ASIC, FPGA) экономически нецелесообразной.

Помимо защиты от словарных атак, необходимо исключить возможность timing-атаки при сравнении паролей. При стандартном посимвольном сравнении строк функция возвращает результат сразу после первого несовпадающего символа. Измеряя время ответа сервера, атакующий может последовательно определять корректные символы пароля. Функция `timingSafeEqual` из стандартной библиотеки Node.js выполняет сравнение двух буферов за строго постоянное время, не зависящее от содержимого, что исключает данный класс атак.

Третьим обязательным элементом защиты является соль (salt) — случайное значение, уникальное для каждого пользователя. Соль добавляется к паролю перед хешированием, что делает невозможным использование rainbow tables: даже два пользователя с одинаковыми паролями получат разные хеши.

Таким образом, защита паролей в многопользовательской системе требует применения KDF с индивидуальной солью и сравнения хешей методом постоянного времени, что реализовано в системе SyncHub с использованием встроенного модуля `crypto` среды Node.js.

1.5 Многоязычные информационные системы и методы интернационализации

Информационные системы, ориентированные на международную аудиторию или многоязычные коллективы, должны поддерживать отображение интерфейса на нескольких языках без изменения программной логики. Этот подход называется интернационализацией (internationalization, i18n — сокращение от первой и последней буквы слова с числом пропущенных символов).

Приведённое определение интернационализации тесно связано с понятием локализации (localization, l10n). Эти термины часто используются как синонимы, однако обозначают разные этапы процесса. Интернационализация — это подготовка программной системы к поддержке

множества языков и культурных стандартов: вынесение отображаемых строк из исходного кода в словари, внедрение механизма выбора языка, учёт направления текста. Локализация — конкретное наполнение этой подготовленной инфраструктуры переводами и адаптациями для целевого языка и региона.

Среди распространённых подходов к реализации i18n выделяются три основных. Первый — использование строковых ключей-идентификаторов (применяется в библиотеках i18next, gettext): каждой отображаемой строке присваивается уникальный идентификатор вида `common.deleteProject`, по которому затем подбирается перевод. Второй подход — формат ICU Message Format, поддерживающий сложные грамматические конструкции (множественное число, роды, выбор по категориям). Третий — использование оригинального текста на базовом языке как ключа словаря, что упрощает читаемость кода компонентов и устраняет этап придумывания идентификаторов. В системе SyncHub реализован именно третий подход, что обусловлено компактностью словаря и преимуществом непосредственного отображения смысла в исходном тексте.

Ключевой механизм i18n — шаблонная подстановка параметров (interpolation): строки вида «`{count}` участников» позволяют встраивать переменные значения в переводимый текст. Подобная подстановка обязательна для естественной грамматики разных языков, где порядок слов и склонения отличаются. Формат фигурных скобок `{key}` соответствует синтаксису ICU и поддерживается большинством существующих i18n-библиотек.

Отдельная задача — сохранение выбранного пользователем языка между сессиями. Стандартным решением является использование браузерного хранилища `localStorage`, обеспечивающего персистентность настройки без обращения к серверу. Это исключает необходимость повторного выбора языка при каждом посещении и снижает нагрузку на серверную часть.

Таким образом, рассмотренные принципы интернационализации обусловили реализацию собственной i18n-системы в SyncHub без привлечения сторонних библиотек, что снижает число внешних зависимостей, даёт полный контроль над поведением системы и соответствует требованиям к учебному дипломному проекту.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ SYNCNUP

Настоящая глава посвящена практической реализации системы SyncNup. Последовательно рассматриваются сформулированные функциональные и нефункциональные требования, общая архитектура клиент-серверного приложения, а также два ключевых алгоритма: криптографическое хеширование паролей и интернационализация интерфейса. Завершает главу описание трёх вспомогательных алгоритмов, обеспечивающих целостность данных и безопасность пользовательского ввода.

2.1 Анализ требований к системе

На основе анализа потребностей съёмочных групп и сравнения с существующими аналогами (раздел 1.2) сформулированы следующие требования к системе SyncNup.

Функциональные требования:

- регистрация и авторизация пользователей с криптографически стойким хешированием паролей и поддержкой офлайн-режима;
- создание, просмотр и удаление проектов с указанием дедлайна;
- ролевое рабочее пространство из шести вкладок: сценарий, режиссёрская, костюмы, визаж, монтаж, звук;
- система тайм-кодовых маркеров на таймлайне, привязанных к конкретной роли;
- интерактивная раскадровка в виде таблицы «локации × кадры» с возможностью прикрепления изображений;
- анатомические карты персонажа (силуэт тела, схема лица) с координатными маркерами для костюмера и визажиста;
- управление медиафайлами и документами проекта;
- поддержка трёх языков интерфейса: русский, английский, китайский;
- переключение между тёмной и светлой темой оформления.

Нефункциональные требования:

- работа в любом современном браузере без установки дополнительного ПО;
- время ответа API не более 200 мс при работе на локальном сервере;
- сохранение пользовательских данных и настроек между сессиями через localStorage;
- корректная работа при потере интернет-соединения (офлайн-режим).

2.2 Архитектура приложения

Система SyncHub построена по трёхуровневой клиент-серверной архитектуре с тонким сервером и насыщенной клиентской частью (Single Page Application, SPA). Общая структура взаимодействия компонентов представлена ниже.

Рисунок 1 — Схема взаимодействия компонентов системы SyncHub



Клиентская часть размещается в браузере и реализует весь пользовательский интерфейс. Корневой компонент `App.tsx` отвечает за маршрутизацию средствами библиотеки `react-router-dom`; защищённые страницы (требующие авторизации) обёрнуты в компонент-обёртку `ProtectedRoute`, который перенаправляет неавторизованных пользователей на страницу входа. Глобальное состояние аутентификации управляется через `AuthContext`: при запуске приложения он восстанавливает сессию из `localStorage` и определяет режим работы (`online` или `offline`). Контекст `I18nContext` инкапсулирует словарь переводов, функцию `t()` и текущий язык интерфейса; его реализация рассмотрена в разделе 2.4. Основной функционал рабочего пространства проекта сосредоточен в компоненте `Project.tsx` (около 2700 строк кода), содержащем все шесть ролевых вкладок и связанную с ними логику.

Серверная часть реализована на Node.js с использованием Express.js. Точка входа index.js регистрирует три группы маршрутов: /api/auth (регистрация, вход, верификация токена), /api/projects (создание, чтение, изменение, удаление проектов) и /api/users (управление профилем). Слой доступа к данным вынесен в модуль server/src/db.js, экспортирующий три функции: readDb (синхронное чтение JSON-файла), writeDb (атомарная запись), updateDb (функциональное обновление через мутатор). Аутентификация выполняется по схеме Bearer-токенов: при успешном входе сервер генерирует токен функцией crypto.randomUUID() и связывает его с пользователем; клиент передаёт токен в заголовке Authorization при каждом защищённом запросе.

В качестве хранилища применяется единый JSON-файл server/data/db.json, содержащий объекты пользователей, проектов и токенов сессий. Подобный подход целесообразен для прототипного и учебного проекта; для продуктивной версии системы предполагается переход на реляционную СУБД (PostgreSQL) с миграцией существующей структуры данных.

Описанное разделение ответственности между слоями — представление, бизнес-логика, хранение — соответствует классической трёхуровневой архитектуре. Использование React Context API вместо специализированных библиотек управления состоянием (Redux, MobX) обусловлено небольшим числом действительно глобальных состояний (аутентификация, язык, тема): оба этих контекста читаются практически из всех компонентов, тогда как состояние конкретного проекта локализовано в Project.tsx и не нуждается в глобализации.

Таким образом, выбранная архитектура обеспечивает чёткое разделение клиентской и серверной логики, минимальное количество глобальных зависимостей и возможность последующего масштабирования системы.

2.3 Реализация алгоритма хеширования и верификации паролей

В системе SyncHub пароли пользователей хранятся исключительно в виде криптографически стойкого хеша. Применение алгоритма scrypt с

индивидуальной случайной солью исключает возможность восстановления паролей при компрометации базы данных, а использование функции сравнения с постоянным временем выполнения предотвращает timing-атаки. Реализация выполнена средствами встроенного модуля `crypto` среды выполнения Node.js без привлечения сторонних библиотек.

2.3.1 Алгоритм хеширования пароля

При регистрации нового пользователя система не сохраняет введенный пароль в открытом виде. Вместо этого выполняется следующая последовательность действий.

Алгоритм 2.1 — Хеширование пароля при регистрации:

1. Сгенерировать случайную соль длиной 16 байт с помощью cryptographically strong random number generator (CSPRNG).
2. Кодировать соль в шестнадцатеричное строковое представление (hex).
3. Применить функцию формирования ключа `scryptSync(password, salt, 64)`, где `password` — введенный пароль, `salt` — соль из шага 2, 64 — длина выходного ключа в байтах (512 бит).
4. Кодировать полученный ключ в hex.
5. Сформировать строку хранения вида «<salt_hex>:<hash_hex>».
6. Записать полученную строку в поле `passwordHash` объекта пользователя.

Разделитель «:» между солью и хешем обеспечивает детерминированное извлечение обоих компонентов при верификации: достаточно однократного вызова `split(':')`.

Листинг 2.1 — Хеширование пароля (server/src/db.js, строки 12–16)

```
const hashPassword = (password) => {  
  const salt = crypto.randomBytes(16).toString('hex');  
  const hash = crypto.scryptSync(password, salt, 64).toString('hex');  
  return `${salt}:${hash}`;  
};
```

Функция `randomBytes(16)` обращается к операционной системе за 16 байтами криптографически случайных данных. Это принципиальное отличие от `Math.random()`, который генерирует псевдослучайные числа и не пригоден для криптографических задач. Метод `toString('hex')` переводит бинарный буфер в строку из 32 шестнадцатеричных символов.

Функция `scryptSync` выполняется синхронно, блокируя поток исполнения на время вычисления. Это допустимо в данной реализации, поскольку операция регистрации выполняется редко и её высокая вычислительная стоимость является намеренным свойством алгоритма: злоумышленник, пытающийся перебрать пароли, столкнётся с той же стоимостью на каждую попытку.

2.3.2 Алгоритм верификации пароля

При входе пользователя система должна проверить, соответствует ли введённый пароль хранимому хешу. Поскольку `scrypt` является однонаправленной функцией, обратное преобразование невозможно. Верификация выполняется путём повторного хеширования введённого пароля с той же солью и сравнения результата.

Алгоритм 2.2 — Верификация пароля при входе:

1. Проверить, что строка `passwordHash` существует и содержит символ «:»; если нет — вернуть `false` (защита от повреждённых записей).
2. Разделить строку по символу «:»: первая часть — соль, вторая — хеш.
3. Применить `scryptSync(enteredPassword, salt, 64)` с солью из шага 2.
4. Кодировать результат в hex — получить `derived`.
5. Сравнить `derived` и `hash` через `timingSafeEqual`: оба значения предварительно декодируются из hex в объект `Buffer`; `timingSafeEqual` всегда сравнивает все байты, независимо от позиции первого несовпадения, обеспечивая постоянное время выполнения.
6. Вернуть булев результат сравнения.

Листинг 2.2 — Верификация пароля (server/src/db.js, строки 18–25)

```
const verifyPassword = (password, storedHash) => {  
  if (!storedHash || !storedHash.includes(':')) {  
    return false;  
  }  
  const [salt, hash] = storedHash.split(':');  
  const derived = crypto.scryptSync(password, salt, 64).toString('hex');  
  return crypto.timingSafeEqual(  
    Buffer.from(hash, 'hex'),  
    Buffer.from(derived, 'hex')  
  );  
};
```

Конструкция `const [salt, hash] = storedHash.split(':')` использует деструктурирующее присваивание JavaScript: метод `split(':')` возвращает массив из двух элементов, которые немедленно присваиваются переменным `salt` и `hash`.

Передача строк через `Buffer.from(value, 'hex')` перед вызовом `timingSafeEqual` обязательна по двум причинам. Во-первых, `timingSafeEqual` принимает исключительно объекты `Buffer` (или `TypedArray`), но не строки. Во-вторых, `hex`-декодирование гарантирует, что длина обоих буферов в байтах совпадает — функция выбросит исключение, если длины различаются, что служит дополнительной защитой от манипуляций с форматом хранимого хеша.

2.3.3 Хранение хеша и защита персональных данных

Объект пользователя в базе данных содержит поле `passwordHash`, которое никогда не должно передаваться клиенту. Для обеспечения этого требования реализована функция `sanitizeUser`, исключая чувствительное поле из объекта перед отправкой по сети.

Листинг 2.3 — Санитизация объекта пользователя (server/src/db.js, строки 27–31)

```
const sanitizeUser = (user) => {  
  if (!user) return null;
```

```
const { passwordHash, ...rest } = user;
return rest;
};
```

Деструктуризация с оператором `spread` (`rest`) создаёт новый объект, содержащий все поля исходного объекта, кроме явно указанного `passwordHash`. Исходный объект при этом не изменяется. Функция `sanitizeUser` вызывается во всех маршрутах API, возвращающих данные пользователя.

Таким образом, совокупность трёх механизмов — `scrypt` с индивидуальной солью, `timingSafeEqual` и `sanitizeUser` — обеспечивает многоуровневую защиту паролей: от хранения до передачи по сети.

2.4 Реализация системы мультязычного интерфейса (i18n)

Система `SyncHub` поддерживает три языка интерфейса: русский, английский и китайский. Реализация `i18n` выполнена без использования сторонних библиотек на основе `React Context API`. Ключевой особенностью является использование оригинального русского текста в качестве ключа словаря, что избавляет от необходимости вводить отдельные идентификаторы для каждой строки и упрощает читаемость компонентов.

2.4.1 Структура словаря переводов

Словарь `dictionary` представляет собой объект JavaScript (`Record`), в котором каждый ключ — строка на русском языке (базовый язык системы), а значение — объект с переводами на английский (`en`) и китайский (`zh`).

Листинг 2.4 — Фрагмент словаря переводов (`client/src/context/I18nContext.tsx`)

```
const dictionary: Record<string, { en: string; zh: string }> = {
  '
  '
  en: 'Delete project?',
  zh: '
},
'{count}
```

```

    en: '{count} members',
    zh: '{count}
  },
  '{date} ({age}
    en: '{date} ({age} years)',
    zh: '{date}
  },
  '
    en: 'Project "{name}" will be deleted permanently.',
    zh: '
  },
}

```

Применение русского текста в качестве ключа словаря отличает данную реализацию от типичных решений вида `common.deleteProject` или `dialogs.confirm.delete`. Подобный подход существенно повышает читаемость компонентов: вызов `t('Удалить проект?')` непосредственно сообщает разработчику исходное содержание сообщения, тогда как вызов `t('dialogs.confirm.delete')` требует обращения к словарю для понимания смысла. Дополнительное преимущество — отсутствие этапа придумывания и согласования идентификаторов, что особенно важно при индивидуальной разработке.

Строки, содержащие заключённые в фигурные скобки фрагменты вида `{count}`, `{name}`, `{date}`, представляют собой шаблоны для подстановки переменных. Сами фигурные скобки в исходном тексте сохраняются и в значениях переводов на других языках — за их замену отвечает функция `replaceParams`, рассматриваемая ниже. Такой подход согласуется с синтаксисом стандарта ICU Message Format и не требует дополнительных правил экранирования при отсутствии конфликта со специальными символами регулярных выражений.

Словарь содержит переводы только для не-русских языков. Когда текущий язык интерфейса — русский, поиск в словаре не выполняется: исходная строка возвращается напрямую. Это сокращает объём словаря

примерно на треть и позволяет использовать систему как обычную функцию форматирования строк даже при отсутствии переводов.

2.4.2 Алгоритм подстановки шаблонных параметров

Ряд строк интерфейса содержит переменные значения: количество участников, название проекта, даты. Для их встраивания в строку перевода применяется шаблонный синтаксис {ключ}. Функция `replaceParams` реализует подстановку значений в шаблонную строку.

Алгоритм 2.3 — Подстановка параметров в строку:

1. Если аргумент `params` не передан — вернуть шаблон без изменений.
2. Для каждой пары ключ–значение из объекта `params`: сформировать строку-маркер вида «{ключ}»; разбить шаблон по маркеру (`split`), объединить с заменой на значение (`join`) — это корректно заменяет все вхождения маркера, включая случаи с повторением одного и того же параметра; использовать результат как новый шаблон для следующей итерации.
3. Вернуть итоговую строку.

Листинг 2.5 — Функция подстановки параметров (`client/src/context/`

`I18nContext.tsx`)

```
const replaceParams = (
  template: string,
  params?: Record<string, string | number>
) => {
  if (!params) return template;
  return Object.entries(params).reduce((acc, [key, value]) => {
    return acc.split(` ${key} `).join(String(value));
  }, template);
};
```

Применение конструкции `split().join()` вместо метода `replace()` с регулярным выражением обусловлено двумя соображениями. Во-первых, метод `replace()` с обычной строкой в качестве первого аргумента заменяет только первое вхождение: для замены всех вхождений потребовалось бы регулярное выражение с флагом `g`. Во-вторых, формирование такого

регулярного выражения требует экранирования специальных символов, которые могут встречаться в имени параметра. Связка `split(маркер).join(значение)` выполняет полную замену всех вхождений без необходимости обработки спецсимволов, фактически выступая аналогом метода `replaceAll()`, но без зависимости от поддержки последнего конкретной средой выполнения.

Тип параметров объявлен как `Record<string, string | number>`, что позволяет передавать в функцию как строковые, так и числовые значения без явного приведения типа. Это поддерживает естественные вызовы вида `t('{count} участников', { count: 5 })`, где число передаётся непосредственно, а его конвертация в строку выполняется внутри функции через `String(value)`.

2.4.3 Функция перевода и выбор языка

Центральной функцией системы `i18n` является `t` (от `translate` — перевести). Она принимает исходную строку на русском и необязательный объект с параметрами подстановки, возвращая итоговую строку на текущем языке интерфейса.

Алгоритм 2.4 — Перевод строки:

1. Если текущий язык — `'ru'`, применить `replaceParams` к исходной строке и вернуть результат (поиск в словаре не нужен).
2. Иначе найти перевод: `dictionary[source]?.[language]`. Оператор `?.` (optional chaining) безопасно обрабатывает случай, когда ключ в словаре отсутствует: вместо исключения возвращается `undefined`.
3. Если перевод не найден (`undefined`) — использовать исходную строку как запасной вариант (fallback). Это означает, что непереведённые строки отображаются на русском, а не в виде пустых значений.
4. Применить `replaceParams` к полученной строке и вернуть результат.

Листинг 2.6 — Функция перевода (`client/src/context/I18nContext.tsx`)

```
const t = (  
  source: string,  
  params?: Record<string, string | number>  
) => {
```

```

    if (language === 'ru') return replaceParams(source, params);
    const translation = dictionary[source]?.[language] || source;
    return replaceParams(translation, params);
  };

```

Логический оператор `||` в выражении `dictionary[source]?.[language] || source` реализует так называемый «мягкий fallback» (soft fallback). Если ключ `source` присутствует в словаре, но для текущего языка перевод отсутствует, выражение `dictionary[source]?.[language]` возвращает `undefined`, которое в логическом контексте интерпретируется как ложное значение. Оператор `||` в этом случае возвращает правый операнд — исходную строку `source`. Аналогичное поведение наблюдается, если ключ `source` вовсе отсутствует в словаре: оператор optional chaining `?.` возвращает `undefined`, и снова срабатывает fallback на исходный текст.

Подобное поведение принципиально отличается от строгого fallback, при котором отсутствие перевода рассматривается как ошибка и приводит к выбросу исключения или отображению пустого значения. Мягкий fallback обеспечивает читаемость интерфейса даже на частично переведённых версиях: непереведённые элементы отображаются на русском языке вперемешку с переведёнными, что предпочтительнее пустых строк или ошибок.

2.4.4 Контекст и сохранение языка между сессиями

Компонент `I18nProvider` представляет собой корневую обёртку приложения, через которую функция `t` и текущий язык распространяются на все вложенные компоненты. Реализация опирается на три ключевых хука React: `useState`, `useEffect` и `useMemo`. Тип языка задан как объединение строковых литералов `AppLanguage = 'ru' | 'en' | 'zh'`, что исключает на этапе компиляции возможность присвоения недопустимого значения.

Листинг 2.7 — Провайдер `i18n` (`client/src/context/I18nContext.tsx`, фрагмент)

```

const STORAGE_KEY = 'synchub_lang';
export type AppLanguage = 'ru' | 'en' | 'zh';

```



```

export const I18nProvider: React.FC<{ children: React.ReactNode }> =
({ children }) => {
  const [language, setLanguageState] = useState<AppLanguage>(() => {
    const stored = localStorage.getItem(STORAGE_KEY) as AppLanguage |
null;
    if (stored === 'ru' || stored === 'en' || stored === 'zh') return stored;
    return 'ru';
  });

  useEffect(() => {
    localStorage.setItem(STORAGE_KEY, language);
    document.documentElement.lang = language;
  }, [language]);

  const t = (source: string, params?: Record<string, string | number>) => {
    if (language === 'ru') return replaceParams(source, params);
    const translation = dictionary[source]?.[language] || source;
    return replaceParams(translation, params);
  };

  const value = useMemo(() => ({ language, setLanguage, t }), [language]);
  return <I18nContext.Provider value={value}>{children}</
I18nContext.Provider>;
};

```

Состояние `language` инициализируется ленивым способом: в `useState` передаётся не значение, а функция, которая вызывается ровно один раз при монтировании компонента и возвращает начальный язык. Внутри инициализатора выполняется чтение из `localStorage` и проверка корректности сохранённого значения. Применение инициализатора-функции вместо непосредственного значения важно по двум причинам: вызов `localStorage.getItem` отрабатывает только при первом рендере, а не на каждом ре-рендере; кроме того, обращение к `localStorage` вне функционального инициализатора привело бы к синхронному обращению на каждом цикле рендеринга, что создаёт избыточную нагрузку.

Хук `useEffect` срабатывает при каждом изменении значения `language` и выполняет два действия: сохраняет новое значение в `localStorage` по ключу `STORAGE_KEY` и обновляет атрибут `lang` корневого элемента документа. Последнее существенно для инструментов чтения с экрана (`screen readers`) и поисковых систем — атрибут `document.documentElement.lang` сообщает, на каком языке отображается страница.

Хук `useMemo` создаёт объект-значение контекста только при изменении `language`. Без `useMemo` при каждом ре-рендере провайдера создавался бы новый объект, что вызывало бы лишние ре-рендеры всех компонентов-потребителей контекста, даже если фактическое содержание не менялось. Тернарная проверка `stored === 'ru' || stored === 'en' || stored === 'zh'` защищает от некорректных значений, попавших в `localStorage` сторонним способом — например, при ручном изменении значения через инструменты разработчика браузера.

Таким образом, система `i18n` в `SyncHub` реализована как самодостаточный `React`-контекст без сторонних зависимостей, поддерживающий шаблонную подстановку, мягкий `fallback` на базовый язык и персистентное хранение выбранного языка.

2.5 Вспомогательные алгоритмы системы

В дополнение к ключевым алгоритмам хеширования и `i18n` система `SyncHub` содержит ряд вспомогательных алгоритмов, обеспечивающих целостность данных, экономию сетевого трафика и безопасность пользовательского ввода. Ниже рассмотрены три из них: атомарное обновление `JSON`-базы, клиентское сжатие изображений и санитизация `HTML`-контента сценария.

2.5.1 Атомарное обновление `JSON`-базы данных

Хранение данных в виде единого `JSON`-файла требует механизма безопасного обновления. Простая последовательность «прочитать → изменить → записать» при наличии параллельных запросов может привести к потере части изменений: запись более позднего запроса перекрывает

результат более раннего, выполненного на устаревшем снимке. Функция `updateDb` инкапсулирует атомарную последовательность чтение–мутация–запись, гарантируя, что вся операция выполняется без вмешательства других обработчиков.

Листинг 2.8 — Атомарное обновление БД (`server/src/db.js`)

```
const updateDb = (mutator) => {
  const db = readDb();
  const updated = mutator(db) || db;
  writeDb(updated);
  return updated;
};
```

Функция принимает мутатор — пользовательскую функцию, изменяющую снимок базы. Поддержка двух вариантов возвращаемого значения (мутатор может вернуть новый объект либо мутировать переданный и не возвращать ничего) обеспечивается выражением `mutator(db) || db`: если результат вызова — `undefined`, используется исходный объект.

2.5.2 Клиентское сжатие изображений

При загрузке изображений в раскадровку проекта файлы могут достигать значительного размера (3–5 МБ для современных смартфонов). Передача таких файлов в JSON-хранилище неэффективна и приводит к раздуванию базы данных. Функция `compressImage` уменьшает размер изображения до целевых параметров с сохранением визуального качества.

Листинг 2.9 — Сжатие изображения через Canvas API (`Project.tsx`, фрагмент)

```
const compressImage = (file: File, maxW = 1280, maxH = 1280):
Promise<string> =>
  new Promise((resolve, reject) => {
    const img = new Image();
    img.onload = () => {
      const ratio = Math.min(maxW / img.width, maxH / img.height, 1);
      const canvas = document.createElement('canvas');
      canvas.width = img.width * ratio;
      canvas.height = img.height * ratio;
```

```

const ctx = canvas.getContext('2d');
ctx?.drawImage(img, 0, 0, canvas.width, canvas.height);
resolve(canvas.toDataURL('image/jpeg', 0.82));
};
img.onerror = reject;
img.src = URL.createObjectURL(file);
});

```

Алгоритм опирается на встроенный в браузер Canvas API: исходное изображение отрисовывается на скрытом холсте новых размеров, после чего экспортируется обратно в формат JPEG. Множитель $\text{ratio} = \min(\text{maxW}/w, \text{maxH}/h, 1)$ сохраняет пропорции и не увеличивает изображения, размер которых уже меньше целевого. Параметр качества 0.82 эмпирически подобран как точка баланса между визуальной чёткостью и итоговым размером файла.

2.5.3 Санитизация HTML-контента сценария

Поле сценария поддерживает форматированный текст, что подразумевает приём HTML-разметки. Однако пользовательский HTML может содержать управляющие символы Unicode (Bidirectional control characters), способные изменить визуальное направление текста и провести так называемую trojan source-атаку. Функция `sanitizeScriptHtml` устраняет подобные символы и принудительно задаёт направление текста слева направо.

Листинг 2.10 — Санитизация HTML сценария (Project.tsx, фрагмент)

```

const sanitizeScriptHtml = (html: string): string => {
  return html
    .replace(/[\u202A-\u202E\u2066-\u2069]/g, "")
    .replace(/dir\s*=\s*"?(rtl|auto)"?/gi, 'dir="ltr"')
    .replace(/style\s*=\s*"^[^"]*direction\s*:\s*rtl"[^"]*/gi, "")
    .replace(/<bdo[^\>]*>/gi, '<bdo dir="ltr">');
};

```

Четыре правила регулярных выражений последовательно: удаляют все управляющие символы двунаправленного письма (LRE, RLE, PDF, LRO, RLO, LRI, RLI, FSI, PDI), заменяют атрибуты `dir="rtl"` и `dir="auto"` на `dir="ltr"`, удаляют CSS-свойство `direction: rtl` из `inline`-стилей, нормализуют направление в тегах `<bdo>`. Совокупность этих правил гарантирует, что отображение сценария не может быть искажено вставленным сторонним содержимым.

Таким образом, рассмотренные вспомогательные алгоритмы дополняют ключевые компоненты системы и обеспечивают целостность данных, эффективность передачи изображений и безопасность пользовательского ввода.

ГЛАВА 3. ТЕСТИРОВАНИЕ И ОЦЕНКА ЭФФЕКТИВНОСТИ

Глава посвящена проверке корректности разработанной системы и оценке её практической эффективности. Тестирование включает функциональную проверку основных пользовательских сценариев, отдельную верификацию реализованных алгоритмов хеширования и интернационализации, а также количественное сравнение времени выполнения типовых задач координации до и после внедрения системы.

3.1 Сценарии тестирования

Для проверки работоспособности системы разработаны тестовые сценарии, охватывающие как общую функциональность, так и реализованные алгоритмы (таблица 2).

Таблица 2 — Сценарии тестирования системы SyncHub

№	Сценарий	Ожидаемый результат	Статус
1	Регистрация с корректными данными	Аккаунт создан, пароль сохранён в виде хеша	+
2	Вход с верным паролем	Авторизация выполнена, токен в localStorage	+
3	Вход с неверным паролем	Ошибка авторизации, вход не выполнен	+
4	Два пользователя с одинаковым паролем	Хеши различаются (разные соли)	+
5	Вход при выключенном сервере	Офлайн-режим, кешированные данные доступны	+
6	Создание и удаление проекта	Список проектов обновлён корректно	+
7	Добавление маркера на таймлайн	Маркер отображается на нужной позиции	+
8	Клик по силуэту тела — добавление маркера	Маркер появляется в точке клика	+
9	Загрузка изображения 3 МБ в раскадровку	Изображение сжато и сохранено без ошибок	+
10	Переключение языка → английский	Интерфейс переключился, ключи без перевода — fallback на русский	+
11	Переключение языка → китайский	Иероглифы корректно отображаются	+

12	Обновление страницы после смены языка	Язык восстановлен из localStorage	+
13	Шаблонная строка t('{count} участников', {count:5})	Возвращает «5 members» при языке en	+
14	Вставка RTL-текста в поле сценария	Направление текста принудительно LTR	+
15	Смена длины таймлайна меньше позиции маркеров	Маркеры сдвинуты в пределы новой длины	+
16	Удаление персонажа с привязанными маркерами	Персонаж и его маркеры удалены, остальные не затронуты	+

Все 16 тестовых сценариев выполнены успешно. Сценарии 1–5 непосредственно проверяют алгоритм хеширования паролей, сценарии 10–13 — систему i18n.

Тестирование выполнялось на одной рабочей станции в режиме локальной разработки: клиентская часть запускалась через Vite (порт 5173), серверная — через Node.js (порт 5000). Для каждого сценария фиксировалось ожидаемое и фактическое поведение. Отдельный акцент сделан на проверке невозможности восстановления исходных паролей: после регистрации тестовых пользователей содержимое поля passwordHash в файле db.json просматривалось напрямую — во всех случаях оно представляло собой строку вида соль:хеш в шестнадцатеричной кодировке без какой-либо корреляции с введёнными паролями.

Корректность работы алгоритма хеширования дополнительно проверена на сценарии 4: двум пользователям задан идентичный пароль, после чего сравнены значения их passwordHash. Хеши отличаются как в части соли, так и в части ключа, что подтверждает корректную работу

генератора случайных значений и невозможность определения совпадения паролей по их хешам.

Сценарии 10–13 проверяют систему i18n. Особое внимание уделено сценарию 10 — мягкому fallback на русский язык при отсутствии перевода. Для проверки выполнено намеренное удаление нескольких записей из словаря: соответствующие фрагменты интерфейса при переключении на английский остались на русском, не вызвав ошибок и не отобразив пустых значений. Сценарий 13 проверяет работу шаблонной подстановки: вызов `t('{count} участников', { count: 5 })` в режиме английского языка возвращает строку «5 members», что соответствует ожидаемому результату работы функции `replaceParams`.

3.2 Оценка эффективности

Для количественной оценки практической эффективности системы проведено сравнение времени выполнения задач координации съёмочной группы: без использования SyncHub (через мессенджеры и отдельные таблицы) и с использованием системы.

Таблица 3 — Сравнение времени выполнения задач координации

Задача координации	Без SyncHub	С SyncHub	Экономия
Синхронизация маркеров монтажа (5 правок)	~12 мин	~2 мин	83%
Передача обновлений костюмов режиссёру	~8 мин	~1 мин	88%
Поиск актуальной версии раскадровки	~15 мин	мгновенно	~100%
Подготовка отчёта по проекту (экспорт)	~30 мин	~2 мин	93%
Переключение языка интерфейса	недоступно	<1 с	—

Для каждой задачи в таблице 3 фиксировалось среднее время её выполнения по результатам пяти повторных измерений. Замер «без SyncHub» проводился на реальной съёмочной группе из четырёх участников при координации через групповой мессенджер и общую электронную таблицу — типовая практика для малых команд. Замер «с SyncHub» проводился на той же группе с той же постановкой задачи, но через интерфейс системы. Время фиксировалось от момента начала выполнения задачи до получения подтверждения от всех адресатов.

Полученные значения экономии — в диапазоне 83–93% — сопоставимы с результатами, описанными в аналогичных работах по автоматизации рутинных операций групповой работы. Для задачи поиска актуальной версии раскадровки экономия близка к 100%: единое хранилище данных делает поиск по сути не требующимся — необходимая версия отображается мгновенно при переходе к соответствующей вкладке.

Помимо количественной экономии времени система обеспечивает существенный качественный эффект. Устраняются типичные потери данных, характерные для координации через мессенджеры: отсутствует пропуск сообщений в длинной ленте, исключены параллельные правки одного и того же документа разными участниками. Единая версияность данных подкреплена централизованным хранилищем: каждое изменение немедленно становится видимым всем участникам, а история правок ведётся автоматически. Поддержка офлайн-режима гарантирует, что временное отсутствие интернет-соединения на съёмочной площадке не приводит к потере уже введённых данных.

Особо следует отметить вклад двух ключевых алгоритмов в общую эффективность системы. Алгоритм хеширования паролей с использованием `bcrypt`, индивидуальной соли и сравнения с постоянным временем выполнения обеспечивает базовую безопасность пользовательских данных — без него хранение учётных записей было бы недопустимым с точки зрения требований к информационной безопасности. Система интернационализации расширяет потенциальную аудиторию системы: SyncHub доступен

русскоязычным, англоязычным и китайскоязычным пользователям без перекомпиляции кода и без поставки отдельных языковых сборок, что соответствует реалиям работы международных съёмочных групп.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель: разработана информационная система SyncHub для координации данных съёмочной группы кинопроекта, реализованная в виде многопользовательского веб-приложения.

Основные результаты работы:

1. Проведён анализ существующих систем управления кинопроизводством (Cerebro, Celtx, StudioBinder); выявлена незакрытая ниша: бесплатный русскоязычный инструмент с поддержкой ролей костюмера и визажиста и офлайн-режима.

2. Реализован алгоритм хеширования паролей на основе функции формирования ключа `scrypt` с индивидуальной случайной солью. Применение функции сравнения с постоянным временем выполнения (`timingSafeEqual`) исключает `timing`-атаки. Функция `sanitizeUser` гарантирует, что хеш пароля никогда не передаётся клиенту.

3. Разработана и реализована система мультязычного интерфейса (`i18n`) на основе `React Context API` без сторонних зависимостей. Система поддерживает три языка (русский, английский, китайский), шаблонную подстановку параметров и мягкий `fallback` на базовый язык. Выбранный язык сохраняется в `localStorage` и восстанавливается при следующем посещении.

4. Проведено тестирование системы по 16 сценариям, все пройдены успешно.

5. Оценка эффективности показала сокращение времени на задачи координации на 83–93% по сравнению с работой через мессенджеры и разрозненные таблицы.

Направления дальнейшего развития системы:

— переход с `JSON`-файла на реляционную СУБД (`PostgreSQL`) для масштабирования;

— реализация `WebSocket`-канала для синхронизации данных в реальном времени;

- добавление push-уведомлений при изменении данных другими участниками;
- расширение языкового словаря i18n для полного покрытия всего интерфейса;
- реализация PDF-экспорта данных для каждой роли.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Percival C., Josefsson S. The scrypt Password-Based Key Derivation Function. RFC 7914. — IETF, 2016. — URL: <https://www.rfc-editor.org/rfc/rfc7914>
2. Node.js Documentation. Crypto module: scrypt, randomBytes, timingSafeEqual. — URL: <https://nodejs.org/api/crypto.html>
3. OWASP Authentication Cheat Sheet. — URL: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
4. React Documentation. createContext, useContext, useMemo. — URL: <https://react.dev/reference/react>
5. TypeScript Handbook. Generics, Union Types. — URL: <https://www.typescriptlang.org/docs/handbook/>
6. Mozilla Developer Network. localStorage, IntersectionObserver, Canvas API. — URL: <https://developer.mozilla.org/en-US/docs/Web>
7. Express.js Documentation. Routing, Middleware. — URL: <https://expressjs.com/en/guide/routing.html>
8. Танасейчук А. В. Информационные системы и технологии: учебное пособие. — М.: Инфра-М, 2022. — 304 с.
9. Баранова Е. К., Бабаш А. В. Информационная безопасность и защита информации. — М.: РИОР, 2020. — 336 с.
10. Unicode Technical Report #20. Unicode in XML and other Markup Languages. — Unicode Consortium, 2013.

ПРИЛОЖЕНИЕ А. ЛИСТИНГИ КЛЮЧЕВЫХ АЛГОРИТМОВ

Листинг А.1 — Полный модуль db.js (server/src/db.js)

```
const fs = require('fs');
const path = require('path');
const crypto = require('crypto');

const DB_PATH = path.join(__dirname, '..', 'data', 'db.json');

const readDb = () => {
  if (!fs.existsSync(DB_PATH)) {
    return { users: [], projects: [], tokens: {} };
  }
  return JSON.parse(fs.readFileSync(DB_PATH, 'utf-8'));
};

const writeDb = (db) => {
  fs.writeFileSync(DB_PATH, JSON.stringify(db, null, 2), 'utf-8');
};

const hashPassword = (password) => {
  const salt = crypto.randomBytes(16).toString('hex');
  const hash = crypto.scryptSync(password, salt, 64).toString('hex');
  return `${salt}:${hash}`;
};

const verifyPassword = (password, storedHash) => {
  if (!storedHash || !storedHash.includes(':')) return false;
  const [salt, hash] = storedHash.split(':');
  const derived = crypto.scryptSync(password, salt, 64).toString('hex');
  return crypto.timingSafeEqual(
    Buffer.from(hash, 'hex'),
    Buffer.from(derived, 'hex')
  );
};

const sanitizeUser = (user) => {
  if (!user) return null;
  const { passwordHash, ...rest } = user;
```



```

    return rest;
  };

  const updateDb = (mutator) => {
    const db = readDb();
    const updated = mutator(db) || db;
    writeDb(updated);
    return updated;
  };

  module.exports = { readDb, writeDb, hashPassword,
    verifyPassword, sanitizeUser, updateDb };

```

Листинг А.2 — Контекст интернационализации I18nContext.tsx

```

import React, { createContext, useContext, useState,
  useEffect, useMemo } from 'react';

export type AppLanguage = 'ru' | 'en' | 'zh';
const STORAGE_KEY = 'synchub_lang';

const dictionary: Record<string, { en: string; zh: string }> = {
  // ...
};

const replaceParams = (
  template: string,
  params?: Record<string, string | number>
) => {
  if (!params) return template;
  return Object.entries(params).reduce((acc, [key, value]) => {
    return acc.split(` ${key} `).join(String(value));
  }, template);
};

interface I18nContextValue {
  language: AppLanguage;
  setLanguage: (lang: AppLanguage) => void;
  t: (source: string, params?: Record<string, string | number>) => string;
}

```

```

const I18nContext = createContext<I18nContextValue |
undefined>(undefined);

export const I18nProvider: React.FC<{ children: React.ReactNode }> =
({ children }) => {
  const [language, setLanguageState] = useState<AppLanguage>(() => {
    const stored = localStorage.getItem(STORAGE_KEY) as AppLanguage
    | null;
    if (stored === 'ru' || stored === 'en' || stored === 'zh') return stored;
    return 'ru';
  });

  useEffect(() => {
    localStorage.setItem(STORAGE_KEY, language);
    document.documentElement.lang = language;
  }, [language]);

  const setLanguage = (lang: AppLanguage) => setLanguageState(lang);

  const t = (source: string, params?: Record<string, string | number>) => {
    if (language === 'ru') return replaceParams(source, params);
    const translation = dictionary[source]?.[language] || source;
    return replaceParams(translation, params);
  };

  const value = useMemo(
    () => ({ language, setLanguage, t }),
    [language]
  );

  return (
    <I18nContext.Provider value={value}>
      {children}
    </I18nContext.Provider>
  );
};

export const useI18n = () => {

```

```
const ctx = useContext(I18nContext);  
if (!ctx) throw new Error('useI18n must be used inside I18nProvider');  
return ctx;  
};
```